Alibre Catalog Manager — User Guide

Version 1.1 · David Bolsover · https://github.com/bolsover/AlibreAddOnManager

The **Alibre Catalog Manager** is a Windows desktop tool for people who **write, package and publish** Alibre Design add-ons. It:

- creates new, ready-to-build add-on projects in C# or C++;
- builds them and packs them as `.alibreaddon` package files;
- creates and edits an **add-on catalog** — the list of add-ons that the Alibre Add-On Manager reads and installs from.

> **Online catalog:** a public catalog is published at **http://www.bolsover.com/catalog/index.json**. It is an example of the catalog layout this tool produces (see §7), and the address users enter in the Add-On Manager to install the add-ons it lists.

If you only want to *install* add-ons, you need the Alibre Add-On Manager, not this tool.

The Catalog Manager does not connect to Alibre Design. It reads and writes files and runs MSBuild, so it works whether or not Alibre Design is running.

Contents

1. Requirements
2. Installing
3. The main window
4. Catalog tab
5. New Add-On tab
6. Package & Publish tab
7. Catalog layout
8. Publishing a catalog online
9. Typical workflows
10. Troubleshooting
11. Settings

---

1. Requirements

To run the tool

- 64-bit Windows.
- The **.NET 10 Desktop Runtime (x64)**. Setup checks for it and offers to open the download page (https://dotnet.microsoft.com/download/dotnet/10.0) if it is missing.

To build add-ons (New Add-On and Package & Publish tabs)

- **Alibre Design** installed — generated projects reference its API files.
- **MSBuild**, from Visual Studio, Visual Studio Build Tools or JetBrains Rider.
- For **C#** add-ons: the **.NET Framework 4.8 targeting pack**. Alibre Design runs add-ons on .NET Framework 4.8.
- For **C++** add-ons: Visual Studio or Build Tools with the **Desktop development with C++** workload. Rider's bundled MSBuild cannot build C++ projects.

Editing a catalog needs none of these.

2. Installing

1. Run `AlibreCatalogManager_1.1.exe` and accept the UAC prompt.
2. If the .NET 10 Desktop Runtime is missing, Setup asks what to do:
    - **Yes** — open the download page and continue. Install the runtime before starting the tool.
    - **No** — continue without it.
    - **Cancel** — quit Setup.
3. Choose the install folder (default `C:\Program Files\AlibreCatalogManager`) and finish the wizard.

Start the tool from **Start** → **AlibreCatalogManager**. Uninstall it from Windows **Settings** → **Apps** → **Installed apps**.

3. The main window

The window has three tabs:

Tab	Purpose
Catalog	Open, edit, check and save a catalog folder.
New Add-On	Generate a new add-on project in C# or C++.
Package & Publish	Build a project, pack it as a `.alibreaddon`, and add it to the open catalog.

The tabs work together: a project you generate is selected on **Package & Publish** automatically, and publishing adds the package to the catalog open on the **Catalog** tab.

4. Catalog tab

Toolbar

Button	What it does
New...	Creates an empty catalog in a folder you choose (`index.json`, `addons\`, `packages\`).
Open...	Opens an existing catalog by its `index.json`.
Reload	Discards your edits and reads the catalog again.
Save	Writes your changes. You see a list of the changes first, plus a warning if the catalog has errors.

Button	What it does
Add add-on ▾	Adds a new add-on **From package (.alibreaddon)...** or **From build folder...**.
Add version ▾	Adds a new version of the selected add-on, from a package or a build folder.
Remove from listing / **Re-list**	Hides the add-on from `index.json` (or lists it again) without deleting anything.
Delete...	Deletes the add-on from the catalog.
Check catalog	Checks the catalog the way the Add-On Manager will read it.
Search	Filters the add-on list.

Edits stay in memory until you click Save, and Save writes only the files that changed. If you close the window or open another catalog with unsaved changes, you are asked whether to save them.

Add-on list

Every add-on in the catalog, with its **Name**, **Latest** version, **Publisher** and **Status** — **Listed** (in `index.json`) or **Unlisted** (shown in grey).

Editing details

Select an add-on to edit its details in the panel on the right. Select one of its versions in the **Versions** list below to edit that version.

- A change is applied when you press **Enter** (**Ctrl+Enter** in a multi-line field) or leave the field. **Esc** restores the previous value.
- Grey fields are read-only.
- Each field shows a short description when selected.

Add-on fields

Group	Fields
Listing (index.json)	Id *(read-only)*, Status *(read-only)*, Name, Publisher, Summary (one line shown in the Add-On Manager), Tags, Workspaces, Icon URL, Latest version *(read-only, always the newest version)*
Manifest	Folder name, Publisher URL, Publisher email, License (an SPDX identifier, e.g. `MIT`), Homepage, Description

> **Do not change the folder name after release.** It is the add-on's install folder under `C:\ProgramData\Alibre AddOns`; changing it strands existing installs.

Version fields

Group	Fields
Package	Version *(read-only)*, Package URL, SHA-256 *(read-only)*, Size *(read-only)*
Compatibility	Kind (`integrated`), DLL type (`managed` or `native`), Arch (`x64`, or `anycpu` for managed add-ons), Min / Max Alibre build, Prerequisites
Release	Released (`yyyy-MM-dd`), Release notes

Package URL is relative to `index.json` (for example `packages/MyTool-1.0.0.alibreaddon`), or an absolute `https://` address such as a GitHub release asset.

Min / Max Alibre build is the trailing number of the Alibre Design version, e.g. `29000` for V29. Leave empty for no limit. The Add-On Manager installs the newest version compatible with the user's running build.

Adding add-ons and versions

Use **Add add-on** ▾ for a new add-on and **Add version** ▾ for a new version of the selected one. Either can come from:

- **From package (.alibreaddon)...** — a package built earlier.
- **From build folder...** — a build output folder such as `bin\Release` holding the `.adc`, the DLL and, if it has one, `addon.json`. The tool packs it for you. If the folder has no `addon.json`, one is built from the `.adc` and saved into the folder; check its version, and note that it has no minimum Alibre build.

Each package is **validated** first, with the same checks the Add-On Manager applies. Errors stop the package being added; warnings ask you to confirm. The tool records the package's SHA-256 and size, and copies it into `packages\` when you save.

Removing

Action	Effect
Remove from listing	Removes the add-on from `index.json` only. Its manifest and packages stay, so you can list it again later.
Delete...	Deletes the add-on's index entry and manifest, and optionally package files no other entry uses. Users who already installed the add-on keep it but can no longer update it from this catalog.
Remove version...	Removes one version, optionally deleting its package file.
Rehash	Recomputes a version's SHA-256 and size from its package file. Use this only if you rebuilt the package on purpose: users who downloaded the old file will see a checksum mismatch.

Check catalog

The **Problems** pane at the bottom is refreshed with a quick check whenever you open or save a catalog. **Check catalog** runs a deep check that also opens each package. Double-click a problem to go to its add-on. The checks report, among other things:

- duplicate IDs and folder names shared by different add-ons;
- an ID in `index.json` that does not match the manifest;
- missing or duplicate versions, and an out-of-date latest version;
- missing `sha256` or package URL;
- invalid DLL types or Alibre build ranges;
- packages whose contents do not match the manifest.

Fix every **error** before publishing — the Add-On Manager may reject affected add-ons.

5. New Add-On tab

Generates a new, empty Alibre Design add-on project: the minimum that builds, loads into Alibre Design and can be packaged for a catalog. It has one root menu with one command, which is greyed out on Alibre builds older than the minimum you set.

Fields

Field	Meaning
Language	**C#** — a managed .NET Framework 4.8 DLL. **C++** — a native x64 DLL.
Project name	Identifier for the project, DLL, class names and deploy folder, e.g. `MyTool`.
Display name	The add-on's friendly name, shown by Alibre Design and in the catalog.
Menu text	Text of the add-on's menu on the **Add-Ons** ribbon tab.
Author, **Author link**	Shown in the add-on's details. The link is optional.
Description	Used in the `.adc`, `addon.json`, the assembly and the command tooltip.
Workspace	Where the add-on loads: Always, Part, SheetMetal, Assembly, Drawing or Home. *Always* is usually better than *Part*, which excludes sheet metal. A *Home* add-on is launched from the Home window's add-ons list.
Version	The first version, `major.minor.patch` (default `1.0.0`).
Minimum Alibre build	The add-on greys its command out below this build (default `29000` = V29).
Identifier	A new GUID, unique to this add-on. Click **New** for another.
Alibre Program folder	Alibre Design's `Program` folder, which holds the API files the project references.
Output folder	The project is created in a sub-folder named after the project.
Icon	Click **Colour...** to choose a colour (swatch or `RRGGBB` hex). A placeholder icon is drawn with the display name's initials.
Include deploy\install.ps1 and uninstall.ps1	Adds scripts that copy the build output to `C:\ProgramData\Alibre AddOns\<Name>` and register it with Alibre Design — handy for testing without a catalog.
Add the project to the solution	Adds the project to a `.slnx` solution found above the output folder.

Click **Generate**. When it finishes you can open the project folder, and the new project is selected on the **Package & Publish** tab.

What is generated

Every project contains:

File	Purpose
`<Name>.adc`	Add-on descriptor: identifier, DLL type, workspace, menu text, author.
`<Name>.ico`	The generated icon.
`addon.json`	Package manifest used by the catalog.

File	Purpose
`deploy\install.ps1`, `deploy\uninstall.ps1`	Optional test-install scripts. They ask for administrator permission when needed.

A **C#** project adds `<Name>.csproj`, `AlibreAddOn.cs` (entry points), `<Name>AddOn.cs` (**your code goes here**), `AlibreVersion.cs` (minimum-build check) and `Properties\AssemblyInfo.cs`.

A **C++** project adds `<Name>.vcxproj`, `pch.h` / `pch.cpp`, `AlibreAddOn.cpp` (entry points), `<Name>AddOn.h` / `<Name>AddOn.cpp` (**your code goes here**) and `AlibreVersion.h` / `AlibreVersion.cpp`.

6. Package & Publish tab

Work down the three numbered sections. Progress and build output appear in the **Output** box at the bottom.

1. Project

Field	Meaning
Project	The add-on project (`.csproj` or `.vcxproj`). If it has no `addon.json`, one is built from its `.adc` and DLL.
Version	The version to package. If you change it, `addon.json` and (if present) `Properties\AssemblyInfo.cs` are updated before the build — you are asked first.
Configuration	**Release** or **Debug**.
MSBuild	Found automatically (Visual Studio or Build Tools, with Rider as a fallback). Click **Browse…** to choose one yourself.
Build the project before packaging	Untick to package the existing build output as it is. A changed version always needs a build.

2. Package

Field / button	Meaning
Package folder	Where the package is written, as `<folderName>-<version>.alibreaddon`.
Build & package	Builds the project, copies `addon.json` into the build output, packs the output folder and validates the package. `.pdb` files and older packages are left out.
Cancel	Stops a running build.
Show package	Opens File Explorer at the package.

3. Publish

Field / button	Meaning
Catalog	The catalog open on the Catalog tab. **Open...** or **New...** to choose one here.
Release notes	Optional. Stored with the new version.
Publish to catalog	Adds the package to the catalog — as a new add-on, or a new version of the existing one with the same identifier — and offers to save. The package is copied into the catalog's `packages\` folder.

To host a package somewhere else (for example as a GitHub release asset), publish it, then edit the version's **Package URL** on the Catalog tab.

7. Catalog layout

A catalog is a folder of plain files:

```
<catalog>\
├── index.json              listing: one entry per add-on
├── addons\
│   └── {GUID}.json         per-add-on manifest: folder name, publisher,
│                           description and the list of versions
└── packages\
    └── <FolderName>-<version>.alibreaddon
```

Addresses inside the catalog are resolved relative to `index.json`, exactly as the Add-On Manager resolves them, so the whole folder can be moved or uploaded as is.

Manifests stored on another server (an `http://` or `https://` manifest address) are listed but cannot be edited.

When you save, the tool writes packages first, then manifests, then the index, and deletes files last. The index therefore never points at a file that has not been written yet.

A `.alibreaddon` package is a zip file containing the add-on's `.adc`, DLL, icon and other files, plus `addon.json` at its root.

8. Publishing a catalog online

The Catalog Manager edits a **local** catalog folder. To publish it:

1. Save the catalog and run **Check catalog** — fix any errors.
2. Upload the whole catalog folder (`index.json`, `addons\` and `packages\`) to a web server, keeping the folder structure.
3. Give users the address of `index.json`. They enter it on the **Available** tab of the Add-On Manager.

The public catalog at **http://www.bolsover.com/catalog/index.json** is published this way.

A catalog on a network share works too: users enter the folder path (for example `\\server\share\catalog`) instead of a web address.

> Prefer an `https://` address where your host supports it. Over plain `http://`, the SHA-256 check guards against corrupted downloads but not against deliberate tampering.

After uploading, test the address in the Add-On Manager before announcing it.

9. Typical workflows

Create and publish a new add-on

1. **New Add-On** tab: fill in the fields and click **Generate**.
2. Write your add-on's code in `<Name>AddOn.cs` or `<Name>AddOn.cpp`.
3. **Package & Publish** tab: check the version, click **Build & package**.
4. Open or create a catalog, add release notes, click **Publish to catalog** and save.
5. Upload the catalog folder (§8).

Release a new version

1. **Package & Publish** tab: select the project and enter the new version.
2. **Build & package**, then **Publish to catalog** — it is added as a new version of the existing add-on.
3. Save, **Check catalog**, and upload.

Users get the new version with **Install / Update** in the Add-On Manager.

Add an add-on built elsewhere

On the **Catalog** tab, use **Add add-on ▾ → From package (.alibreaddon)...** (or **From build folder...**), fill in its details, and save.

10. Troubleshooting

Problem	What to do
The tool does not start.	Install the **.NET 10 Desktop Runtime (x64)**.
Choose MSBuild.exe...	No MSBuild was found. Install Visual Studio or Build Tools, or choose `MSBuild.exe` with **Browse...**.
A C++ project does not build.	Use MSBuild from Visual Studio or Build Tools with the **Desktop development with C++** workload; Rider's bundled MSBuild has no C++ support.
A C# project cannot find `AlibreX` or `AlibreAddOn`.	The **Alibre Program folder** chosen when the project was generated no longer exists (for example after an Alibre upgrade). Build with `/p:AlibreProgramDir=<new Program folder>`, or edit the path in the project file.
The version must be major.minor.patch	Enter a version such as `1.2.0`.

Problem	What to do
The package is not valid	The message lists each problem — for example no `.adc` matching the identifier, a missing DLL, a 32-bit DLL, or a C++ DLL without the required exports. Fix the project and package again.
Users report a checksum mismatch.	The package file was replaced after its SHA-256 was recorded. Re-publish it as a new version; use **Rehash** only if you rebuilt it on purpose.
The Add-On Manager does not show a new add-on.	Check it is **Listed**, that you saved, and that the uploaded `index.json` is the new one.

11. Settings

The last opened catalog, the last project and the chosen MSBuild path are saved in `%APPDATA%\AlibreCatalogManager\settings.json`.